

Label Lens — how it works

Scan a barcode, understand the label. A zero-backend web app built for the Enonic AI Hackathon.

Enonic AI Hackathon · September 2026

sgauruseu.github.io/label-lens

Label Lens turns a food label into a score you can argue with: it gets the facts from a public database or from a photograph, and then applies published nutrition rules in code you can read.

There is no backend

The whole application is a static page on GitHub Pages. There is no server of ours anywhere in the picture — no database, no accounts, no logs, no analytics. Everything happens in the browser tab, and the only outbound requests go to two places: Open Food Facts, and a model provider if the user has chosen to use one.

That constraint is not a limitation we worked around; it is the reason the privacy story is simple enough to say out loud in one breath.

Three paths through the app

1. Camera → barcode (no network at all)

Camera frames are decoded by **ZXing**, a library that ships with the app and runs inside the tab. The video never leaves the device. Nothing is uploaded. The only thing that comes out of this step is a number:

```
3017620425035
```

A day-zero check changed this design: the browser's native `BarcodeDetector` API turned out to be **absent in Chrome on Windows**, so ZXing became the primary decoder rather than a fallback. Finding that on day one instead of on stage is the single most valuable hour we spent.

2. Barcode → facts (public database, no key, free)

```
GET https://world.openfoodfacts.org/api/v2/product/3017620425035.json
```

Open Food Facts returns a ready-made record: ingredients, nutrients per 100 g, additives, NOVA processing group, Nutri-Score, net quantity. No API key, no authentication, no cost.

No AI is involved in this path at any point. This is the fast, normal case — and it is the one the live demo leads with.

The data is crowd-sourced, which means it is sometimes wrong. One of our bundled products claims 52 g of fibre per 100 g, which is impossible. The app detects macronutrients that cannot add up, says so on screen, and withholds every bonus for that product while still applying every penalty. Trusting a public dataset is a design decision; trusting it blindly is a bug.

3. Photograph → model (the fallback, and the only place AI appears)

Roughly one product in three has no usable entry in the database — a local brand, a new line, a supermarket own-label. For those, the user photographs the **back of the pack**, where the per-100 g table is printed.

The image is downscaled in the browser to 1400 px, encoded, and sent directly from the user's browser to the provider:

```
POST https://api.anthropic.com/v1/messages
```

The provider returns **only what it could read in that photograph**, as text — a JSON object:

```
{
  "product_name": "Requesón",
  "brands": "Hacendado",
  "quantity": "200 g",
  "nutriments": { "energy-kcal_100g": 139, "fat_100g": 11.6, "sugars_100g": 3.2 }
}
```

It has no database. It is not looking anything up. It is reading a picture of a table.

What the AI does — and what it does not

It does one job: transcription. Printed panel in, structured fields out. The prompt is explicit about the boundaries:

Transcribe only. Do not judge the product and do not add commentary.

Omit any key you cannot read from the photo. Never guess a number.

If values are given per serving, convert to per 100 g.

It does not produce the score. This is the important part, and it is worth saying slowly. The JSON from the model goes through exactly the same function as a database response:

```
JSON → normalizeProduct() → evaluate() → screen
```

`evaluate()` is ordinary code. Every product starts at 100. Points come off for nutrients in the UK FSA amber and red bands, for NOVA processing group, for additives of regulatory concern, for a high share of energy from added sugars; points go on for fibre, protein and fruit-veg-nut content. Every single adjustment is printed on screen as a line item with its reason, and the arithmetic adds up in front of the user.

The model has never seen those thresholds. It does not know a score exists.

Why we drew the line there. A model that scores food gives you an answer you cannot check, cannot test, and cannot defend when someone disagrees. A model that reads a label gives you facts you can verify against the packet in your hand. So the scoring engine is 252 unit tests and 99 % line coverage on pure functions with no network and no DOM; the AI sits outside that boundary, in the adapter layer, where it belongs.

The AI is the typist, not the judge.

Why there is no server-side API key

A static site has nowhere to hide a secret — anything in the bundle is one “view source” away. So the photo feature uses the **user's own** key, entered in Settings and stored in their browser's `localStorage`. It is sent to the chosen provider and nowhere else.

This is an honest trade, and the Settings screen says so rather than hiding it. Anthropic requires an explicit opt-in header for browser-originated calls — `anthropic-dangerous-direct-browser-access` — and the word *dangerous* in that header name is the accurate summary: it is fine for a demo on your own key, and wrong for a public service. The production answer is a thin proxy, which is a second deployment and a second thing that can fail on stage. For a 2.5-day hackathon, we took the honest compromise and documented it.

The architecture that makes this testable

<code>src/core/</code>	pure domain logic – no fetch, no DOM, no storage, no clock
<code>src/adapters/</code>	Open Food Facts, camera, <code>localStorage</code> , model provider
<code>src/ui/</code>	React components, presentation only
<code>src/fixtures/</code>	real captured API responses; power offline mode and the tests

Dependencies point inward only. An ESLint rule **fails the build** if anything in `core/` imports an adapter, a component, React, or touches a browser global. The purity of the domain is enforced, not merely intended — which is what lets the scoring rules be tested exhaustively in plain Node.

LAYER	COUNT	QUESTION IT ANSWERS
Unit tests on <code>src/core</code>	252, 99 % line coverage	Is the rule right?
UI tests (Playwright, production build)	33	Did it reach the screen?
Bundled real API responses	6 products	Does it work with the network unplugged?

The thirty-second version

There's no backend — it's a static page. Point the camera at a barcode and the decoding happens in the tab; the number goes to Open Food Facts, which returns the facts for free. If the product isn't in the database, photograph the back of the pack and a model reads the nutrition table into structured fields. That's all the AI does — it transcribes. The score itself is ordinary code with 252 tests behind it, and every point gained or lost is shown on screen with its reason. The AI is the typist, not the judge.

Label Lens · Enonic AI Hackathon, September 2026 · Product data from Open Food Facts (ODbL) · Nutrient thresholds from the UK Food Standards Agency front-of-pack guidance · Informational only — not medical or dietary advice.